

CS313 Lab Report

Archie Harrodine

March 2024

1 Task 1

1.1 Task 1-1

I successfully managed to boot up the VM! Wouldn't be much else in here otherwise.

1.2 Task 1-2

Command to create the hello world package.

```
$ ros2 pkg create --build-type ament_python r2
```

Code snippet showing the hello world program.

```
1 #! /usr/bin/python
2 import rclpy
3 from rclpy.node import Node
4
5 rclpy.init(args=None)
6 node = Node('printer_node') # Note that there should have no space in node name
7 node.get_logger().info('HELLO WORLD - ROS2')
8 rclpy.spin(node)
9 node.destroy_node()
10 rclpy.shutdown()
```

Commands to run the program with ros2 run and ros2 launch.

```
$ cd ~/ros2_ws
$ colcon build --packages-select r2
$ source install/local_setup.bash
$ ros2 run r2 prt
```

```
$ cd ~/ros2_ws
$ colcon build --packages-select r2
$ source install/local_setup.bash
$ ros2 launch r2 node_launcher.launch.py
```

1.3 Task 1-3

Snippet of your pubsub.launcher.launch.py file.

```
1 #! /usr/bin/python
2 import launch
3 import launch_ros.actions
4
5 def generate_launch_description():
6     return launch.LaunchDescription([
7         launch_ros.actions.Node(
8             package='r2',
9             executable='pub',
10            output='screen',
11            name='custom_py_node_name'
12        ),
13        launch_ros.actions.Node(
14            package='r2',
15            executable='sub',
```

```

16     output='screen',
17     name='custom_py_node_name'
18 )
19
20 ])

```

Commands to launch it with `ros2 launch`.

`ros2 launch r2 pubsub_launcher.launch.py`

Screenshot of your output.

```

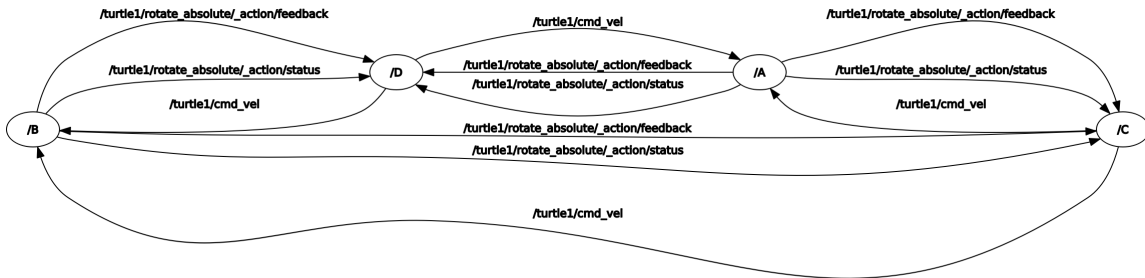
[INFO] [launch]: All log files can be found below /home/dcs/.ros/log/2024-03-20-14-36-59-883152-dcs-KVM-3128
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [pub-1]: process started with pid [3130]
[INFO] [sub-2]: process started with pid [3132]
[pub-1] [INFO] [1710945420.983484845] [custom_py_node_name]: Publishing: "Hello World: 0"
[sub-2] [INFO] [1710945420.984286433] [custom_py_node_name]: I heard: "Hello World: 0"
[pub-1] [INFO] [1710945421.475780401] [custom_py_node_name]: Publishing: "Hello World: 1"
[sub-2] [INFO] [1710945421.475986265] [custom_py_node_name]: I heard: "Hello World: 1"
[pub-1] [INFO] [1710945421.975823446] [custom_py_node_name]: Publishing: "Hello World: 2"
[sub-2] [INFO] [1710945421.976021911] [custom_py_node_name]: I heard: "Hello World: 2"
[pub-1] [INFO] [1710945422.475864599] [custom_py_node_name]: Publishing: "Hello World: 3"
[sub-2] [INFO] [1710945422.476068980] [custom_py_node_name]: I heard: "Hello World: 3"
[pub-1] [INFO] [1710945422.975829981] [custom_py_node_name]: Publishing: "Hello World: 4"
[sub-2] [INFO] [1710945422.976026195] [custom_py_node_name]: I heard: "Hello World: 4"
[pub-1] [INFO] [1710945423.475809035] [custom_py_node_name]: Publishing: "Hello World: 5"
[sub-2] [INFO] [1710945423.476009954] [custom_py_node_name]: I heard: "Hello World: 5"
[pub-1] [INFO] [1710945423.975778127] [custom_py_node_name]: Publishing: "Hello World: 6"
[sub-2] [INFO] [1710945423.975993724] [custom_py_node_name]: I heard: "Hello World: 6"
[pub-1] [INFO] [1710945424.475776784] [custom_py_node_name]: Publishing: "Hello World: 7"
[sub-2] [INFO] [1710945424.476005128] [custom_py_node_name]: I heard: "Hello World: 7"
[pub-1] [INFO] [1710945424.975761995] [custom_py_node_name]: Publishing: "Hello World: 8"
[sub-2] [INFO] [1710945424.975977036] [custom_py_node_name]: I heard: "Hello World: 8"
[pub-1] [INFO] [1710945425.475771911] [custom_py_node_name]: Publishing: "Hello World: 9"
[sub-2] [INFO] [1710945425.475965216] [custom_py_node_name]: I heard: "Hello World: 9"
[pub-1] [INFO] [1710945425.975779816] [custom_py_node_name]: Publishing: "Hello World: 10"
[sub-2] [INFO] [1710945425.975988344] [custom_py_node_name]: I heard: "Hello World: 10"
[pub-1] [INFO] [1710945426.475792942] [custom_py_node_name]: Publishing: "Hello World: 11"
[sub-2] [INFO] [1710945426.476023726] [custom_py_node_name]: I heard: "Hello World: 11"
[pub-1] [INFO] [1710945426.975776174] [custom_py_node_name]: Publishing: "Hello World: 12"
[sub-2] [INFO] [1710945426.976009565] [custom_py_node_name]: I heard: "Hello World: 12"
[pub-1] [INFO] [1710945427.475789123] [custom_py_node_name]: Publishing: "Hello World: 13"
[sub-2] [INFO] [1710945427.475984963] [custom_py_node_name]: I heard: "Hello World: 13"
[pub-1] [INFO] [1710945427.975826192] [custom_py_node_name]: Publishing: "Hello World: 14"
[sub-2] [INFO] [1710945427.975968362] [custom_py_node_name]: I heard: "Hello World: 14"

```

2 Task 2

2.1 Task 2-1

Visualisation of the four nodes: `turtlesim_node` (A and B) and `turtle_teleop_key` (C and D) with `rqt_graph`.

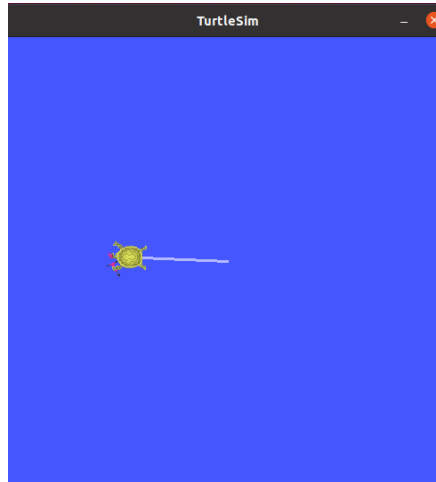


2.2 Task 2-2

Commands to teleport turtlebot to specified location and the respective screenshots.

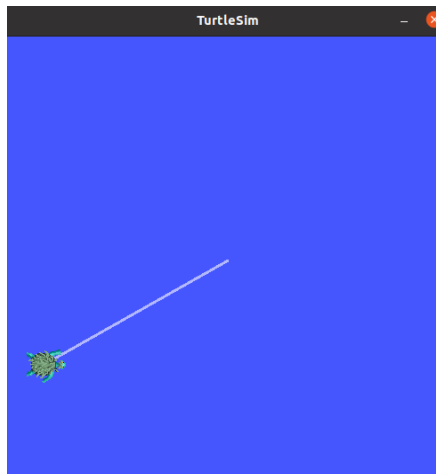
Teleport relative:

```
ros2 service call /turtle1/teleport_relative turtlesim/srv/TeleportRelative '{linear: 2.5, angular: 3.1}'
```



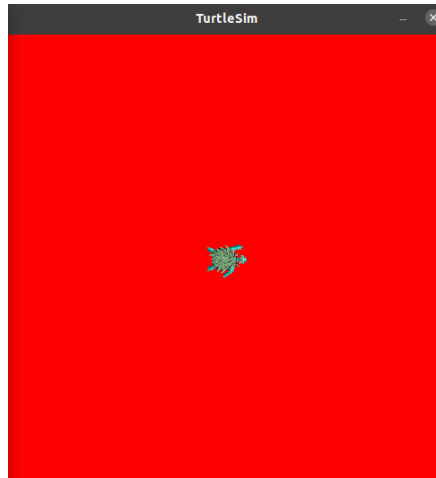
Teleport absolute:

```
ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute '{x: 1.0, y: 3.0, theta: 0.0}'
```



Commands to turn the background red using `param` and the respective screenshot.

```
ros2 param set /turtlesim background_r 255
ros2 param set /turtlesim background_b 0
ros2 param set /turtlesim background_g 0
```

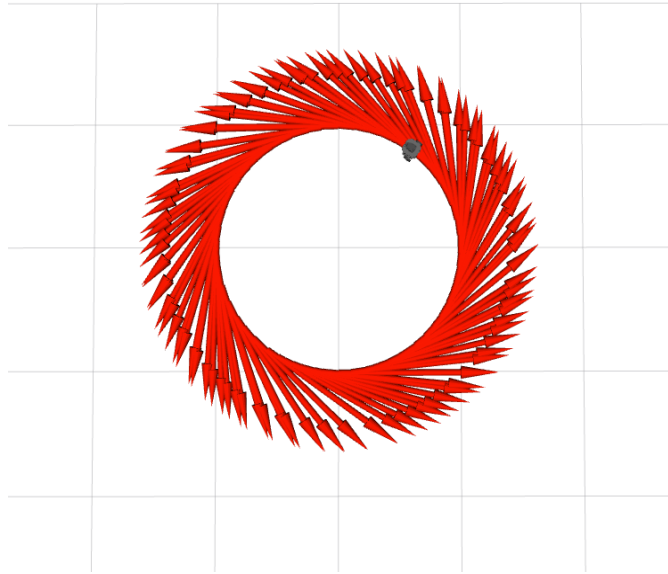


2.3 Task 2-3

Program to drive the TurtleBot in a circle.

```
1  #!/usr/bin/env python
2  import rclpy
3  from rclpy.node import Node
4  from geometry_msgs.msg import Twist
5
6  class Mover(Node):
7      def __init__(self):
8          super().__init__('vel_publisher')
9          self.publisher_ = self.create_publisher(Twist, 'cmd_vel', 10)
10         self.timer_ = self.create_timer(1.0, self.timer_callback)
11         self.i = 0
12         self.get_logger().info("Press CTRL + C to terminate")
13
14     def timer_callback(self):
15         msg = Twist()
16         msg.linear.x = 1.0
17         msg.angular.z = 1.0
18
19         self.publisher_.publish(msg)
20         self.get_logger().info('Publishing: "%s"' % msg)
21
22 def main(args=None):
23     rclpy.init(args=args)
24     mover = Mover()
25     rclpy.spin(mover)
26     mover.destroy_node()
27     rclpy.shutdown()
28
29 if __name__ == '__main__':
30     main()
```

To make it go in a circle, we needed it to go forward constantly, so we gave it a linear speed in the x axis. Then to make it turn, we needed to make it so the robot was going at an angle to their current relative position. Hence we needed to ensure z had some angular component, making it rotate about the z axis. The values didn't really matter so long as they were small enough to be visible on screen, so setting them both to 1 seemed to work. This can be envisioned as moving a vector slightly to the left of where the robot is facing each time, making it go round in a circle.



3 Task 3

3.1 Task 3-1

Program to drive the robot in a square.

```

1 from typing import List
2 import rclpy
3 from rclpy.context import Context
4 from rclpy.node import Node
5 from geometry_msgs.msg import Twist
6 from rclpy.parameter import Parameter
7 from math import pi
8
9 class Mover(Node):
10     def __init__(self, node_name: str = 'mover'):
11         super().__init__(node_name=node_name)
12         self.publisher_ = self.create_publisher(Twist, 'cmd_vel', 10)
13         self.timer_ = self.create_timer(1.0, self.timer_callback) # 1.0s period, 1Hz
14         self.i = 0
15         self.step = 1
16         self.turning = False
17         self.turns = 0
18         msg = Twist()
19         msg.linear.x=1.0
20         msg.angular.z = 0.0
21         self.publisher_.publish(msg)
22         self.get_logger().info('Mover has been initialized.')
23
24     def timer_callback(self):
25         if self.turns < 4:
26             if not self.turning:
27                 self.get_logger().info("Not turning")
28                 if (self.step % 4) == 0:
29                     msg = Twist()
30                     msg.linear.x = 0.0
31                     msg.angular.z = pi/20
32                     self.publisher_.publish(msg)
33                     self.turning=True
34                     self.step=0
35             else:
36                 self.get_logger().info("Turning")
37                 if (self.step % 10) == 0:
38                     msg = Twist()
39                     msg.linear.x = 1.0
40                     msg.angular.z = 0.0
41                     self.publisher_.publish(msg)

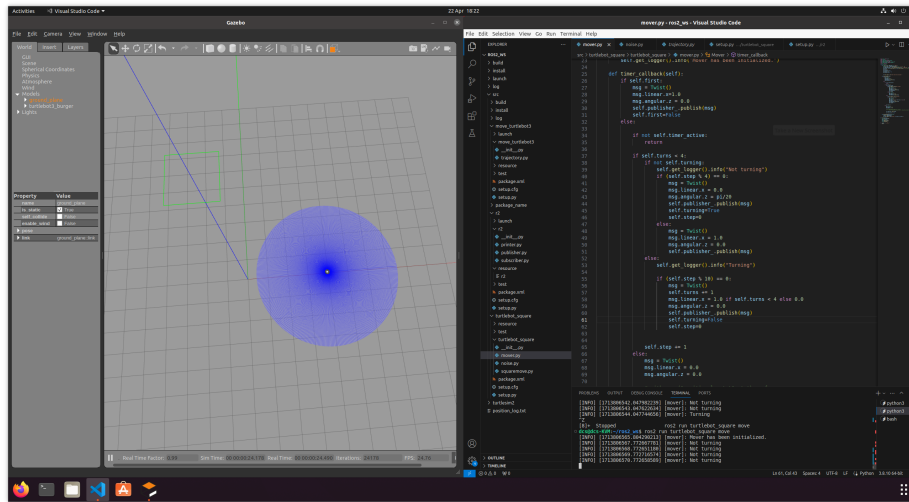
```

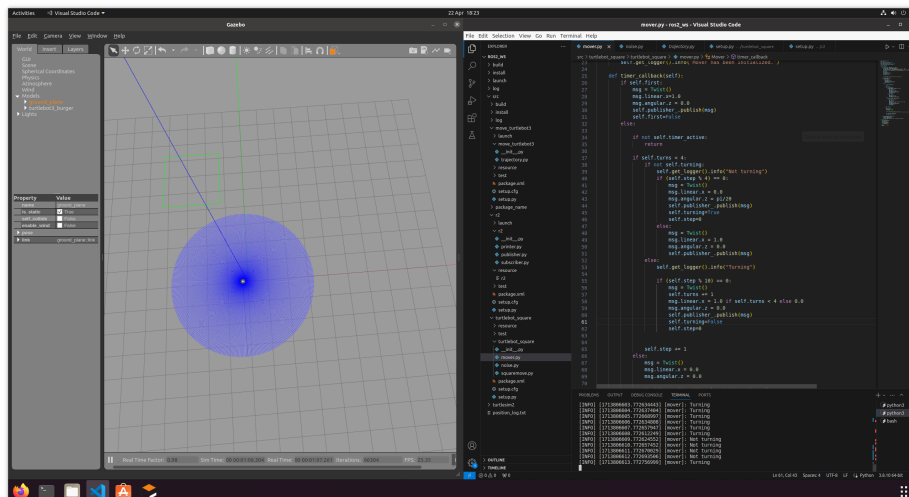
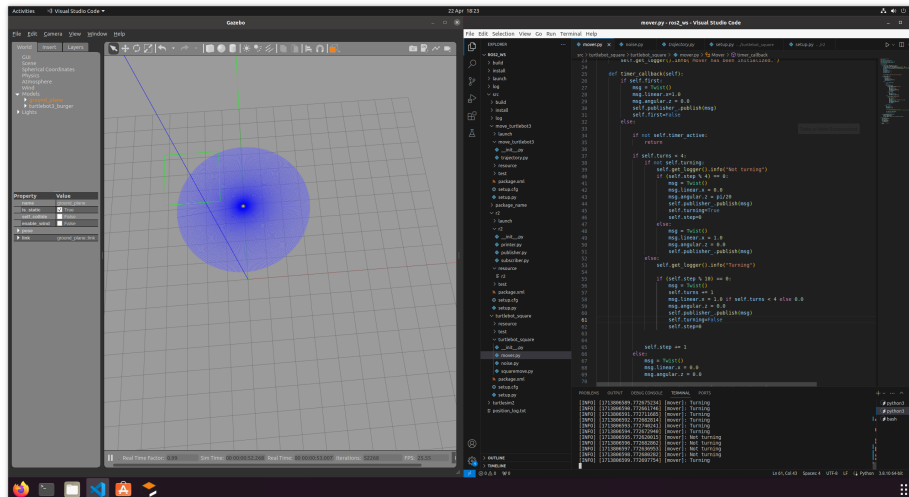
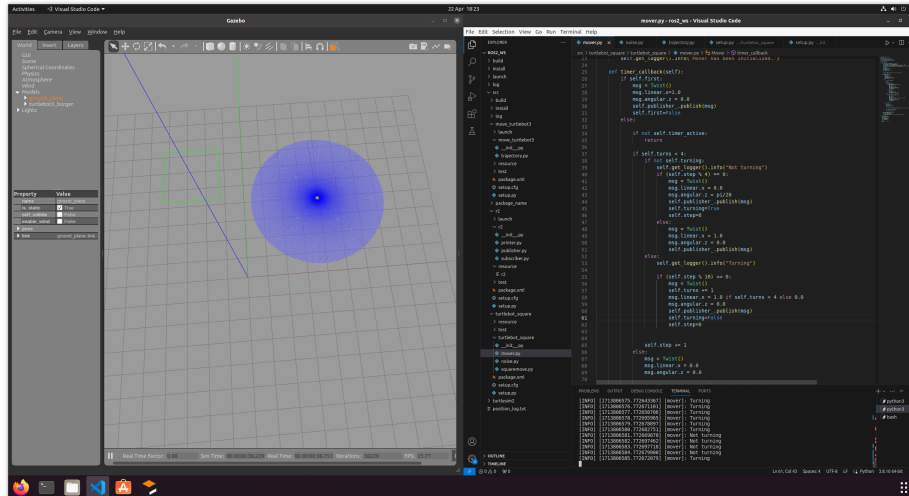
```

42         self.turning=False
43         self.step=0
44         self.turns += 1
45
46         self.step += 1
47     else:
48         msg = Twist()
49         msg.linear.x=0.0
50         msg.angular.z=0.0
51         self.publisher_.publish(msg)
52
53
54 def main(args=None):
55     rclpy.init(args=args)
56     mover = Mover()
57     rclpy.spin(mover)
58     mover.destroy_node()
59     rclpy.shutdown()
60
61 if __name__ == '__main__':
62     main()

```

Working in Gazebo we first align the axes so that the x and y axes lie in the corresponding locations. We then define two states: turning and not turning. Not turning has the robot move linearly straight forward (x axis) for 4 steps, or seconds, with angular speed at 0 everywhere. The turning state works similarly, taking 10 steps to turn 90 degrees to the left. We do it slowly because it loses a lot of accuracy when it is done in one second, so by slowing it down it performs far better. We rotate it about the z axis. We then track the number of turns and when it has turned 4 times, we tell it to stop by setting all speeds across all dimensions to 0.





3.2 Task 3-2

```
1 import numpy as np
2 import rclpy
3 from rclpy.node import Node
4 from geometry_msgs.msg import Twist
5 from rclpy.parameter import Parameter
6
7 class AddMotionNoise(Node):
8     def __init__(self):
9         super().__init__('add_motion_noise')
10
11         self.declare_parameter('linear_noise', value=0.001)
12         self.declare_parameter('angular_noise', value=0.001)
13         self.declare_parameter('topic_name', value='cmd_vel')
14
15         self.linear_noise = self.get_parameter('linear_noise').get_parameter_value().double_value
16         self.angular_noise = self.get_parameter('angular_noise').get_parameter_value().double_value
17         self.topic_name = self.get_parameter('topic_name').get_parameter_value().string_value
18
19         self.get_logger().info('linear_noise: %s' % self.linear_noise)
20         self.get_logger().info('angular_noise: %s' % self.angular_noise)
21         self.get_logger().info('topic_name: %s' % self.topic_name)
22
23         self.publisher_ = self.create_publisher(Twist, self.topic_name, 10)
24         self.subscription_ = self.create_subscription(Twist, 'cmd_vel_2', self.cmd_vel_callback, 10)
25
26         self.subscription_ # prevent unused variable warning
27         self.get_logger().info('AddMotionNoise has been initialized.')
28
29     def cmd_vel_callback(self, msg: Twist):
30         msg.linear.x += np.random.normal(0.0, self.linear_noise)
31         msg.angular.z += np.random.normal(0.0, self.angular_noise)
32
33         self.publisher_.publish(msg)
34
35 def main(args=None):
36     rclpy.init(args=args)
37     add_motion_noise = AddMotionNoise()
38     rclpy.spin(add_motion_noise)
39     add_motion_noise.destroy_node()
40     rclpy.shutdown()
41
42 if __name__ == '__main__':
43     main()
```

Our code works by linking to this second channel that our move script now publishes to, and adding noise to the linear and angular speed in the x and z axis. This message is then published to the original channel which Gazebo finds. We add Gaussian noise using the function that was given, and this skews how the robot moves. The noise for experiment 2 was (0.02, 0.01) and (0.1, 0.05) for (linear x, angular z).

Results:

Experiment 1		Experiment 2		Experiment 3	
x	y	x	y	x	y
-0.237433	-0.036795	0.280984	0.384369	3.660754	-1.018051
-0.071271	0.135553	-0.835704	-0.006449	-1.464195	0.668452
-0.057768	0.309637	1.233326	0.172379	3.447702	-0.535885
-0.360871	0.406785	-0.343344	0.820547	0.663402	1.664127
-0.621765	0.263237	0.669089	-0.479224	4.312932	-1.914546
-0.208546	0.266764	-1.263189	0.786679	-4.091486	-1.401135
-0.218563	0.309677	1.0102266	-0.469711	0.414391	-0.270192
-0.259207	0.219040	-0.359926	0.710330	-3.269972	1.807097
-0.256795	0.260734	-0.641655	-0.387867	0.240905	-3.192771
-0.206780	0.265211	-0.578074	0.332972	3.087353	-1.083177

Table 1: Results.

3.3 Task 3-3

The code to calculate covariance matrix, and the covariance matrices of each experiment. It is trivially symmetric for all of them.

```

1 def covariance_matrix(points):
2     xList = [point[0] for point in points]
3     yList = [point[1] for point in points]
4     xMean = np.mean(xList)
5     yMean = np.mean(yList)
6     xDiff = [x-xMean for x in xList]
7     yDiff = [y-yMean for y in yList]
8     return np.cov(xDiff, yDiff, bias=True)

```

Experiment 1: $\begin{bmatrix} 0.02242622 & -0.00314484 \\ -0.00314484 & 0.01278072 \end{bmatrix}$

The approximate scatter range across the (x, y) axis: (0.14975388101111103, 0.11305182989854697)

Experiment 2: $\begin{bmatrix} 0.60557811 & -0.1896646 \\ -0.1896646 & 0.23397415 \end{bmatrix}$

The approximate scatter range across the (x, y) axis: (0.7781889943556032, 0.4837087448792177)

Experiment 3: $\begin{bmatrix} 8.81643358 & -2.25914025 \\ -2.25914025 & 2.55181332 \end{bmatrix}$

The approximate scatter range across the (x, y) axis: (2.969247982216406, 1.5974396146745384)

There are several methods we could use to reduce the scatter score. Since the robot is being impacted by linear and angular noise, we can look how we mitigate each. Linear noise only significantly affects the turning phase, and this can be reduced by more frequent steps which keep the linear speed at 0. Angular noise meanwhile impacts when we are not turning. It is most impactful when we are moving very fast as we move as sharper angles, hence we can reduce the speed. Additionally we can have more frequent steps which ensure the robot has angular speed at 0. There may be other small changes we can make as well, such as finding methods to reduce the disruption caused by the noise, such as anticipating the noise in advance.

4 Task 4

4.1 Task 4-1

Completed skeleton code of the Controller class.

```

1 class Controller:
2     def __init__(self, P=0.0, D=0.0, set_point=0):
3         self.Kp = P
4         self.Kd = D
5         self.set_point = set_point # reference (desired value)

```

```

6         self.previous_error = 0
7
8     def update(self, current_value):
9         # calculate P_term and D_term
10        error = self.set_point - current_value
11        P_term = self.Kp * error
12        D_term = self.Kd * (error-self.previous_error)
13        self.previous_error = error
14        return P_term + D_term
15
16    def setPoint(self, set_point):
17        self.set_point = set_point
18        self.previous_error = 0
19
20    def setPD(self, P=0.0, D=0.0):
21        self.Kp = P
22        self.Kd = D

```

This was trivial to determine as we effectively just fill in the equation that was given to us.

$$u(t) = k_p e(t) + k_d (e(t) - e(t - \Delta t))$$

The update value is equal to the sum of the proportional term and the derivative term. The way we calculate the error is by taking the point we want to reach, and the current value we are at and finding the difference between them. This is signed so we can correct from either direction. We then use this error in the proportional term by multiplying it by the pre set proportional gain k_p . Meanwhile we get the derivative term by finding the difference between the current error and the error from one step ago. This is a stored value in the skeleton code with previous error, so we find the difference and multiply it by the differential gain k_d .

4.1.1 Task 4-2

Code to drive my turtlebot.

```

1     #!/usr/bin/env python
2     from math import pi, sqrt, atan2, cos, sin
3     import numpy as np
4     import threading
5
6     import rclpy
7     from tf_transformations import euler_from_quaternion
8     from std_msgs.msg import Empty
9     from nav_msgs.msg import Odometry
10    from geometry_msgs.msg import Twist, Pose2D
11
12    class Controller:
13        def __init__(self, P=0.0, D=0.0, set_point=0):
14            self.Kp = P
15            self.Kd = D
16            self.set_point = set_point # reference (desired value)
17            self.previous_error = 0
18
19        def update(self, current_value):
20            # calculate P_term and D_term
21            error = self.set_point - current_value
22            P_term = self.Kp * error
23            D_term = self.Kd * (error-self.previous_error)
24            self.previous_error = error
25            return P_term + D_term
26
27        def setPoint(self, set_point):
28            self.set_point = set_point
29            self.previous_error = 0
30
31        def setPD(self, P=0.0, D=0.0):
32            self.Kp = P
33            self.Kd = D
34
35    class Turtlebot3():
36        def __init__(self):

```

```

37     rclpy.init()
38     self.node = rclpy.create_node("turtlebot3_move_square")
39     self.node.get_logger().info("Press Ctrl + C to terminate")
40     self.vel_pub = self.node.create_publisher(Twist, "cmd_vel", 10)
41     self.rate = self.node.create_rate(10)
42
43     t = threading.Thread(target=rclpy.spin, args=(self.node,), daemon=True)
44     t.start()
45
46     # subscribe to odometry
47     self.pose = Pose2D()
48     self.logging_counter = 0
49     self.trajectory = list()
50     self.odom_sub = self.node.create_subscription(Odometry, "odom", self.odom_callback, 10)
51
52     self.goals = [[4,0], [4,4], [0,4], [0,0]]
53
54     try:
55         self.run()
56     except KeyboardInterrupt:
57         print('Interrupted')
58     finally:
59         # save trajectory to csv file
60         np.savetxt('trajectory.csv', np.array(self.trajectory), delimiter=',')
61         self.node.destroy_node()
62         rclpy.shutdown()
63
64     def run(self):
65         pd = Controller(P=0.8, D=0.6)
66         i = 0
67
68         while i<4:
69             goal = self.goals[i]
70
71             distance = np.sqrt((self.pose.x - goal[0])**2 + (self.pose.y-goal[1])**2)
72             if distance < 0.1:
73                 i+=1
74
75             target = self.target_theta(self.pose.x, self.pose.y, goal[0], goal[1])
76             pd.setPoint(self.normalize_theta(target, goal))
77             msg = Twist()
78             msg.linear.x=0.4
79             msg.angular.z = pd.update(self.normalize_theta(self.pose.theta, goal))
80             self.vel_pub.publish(msg)
81             self.node.get_logger().info(str(distance))
82
83             msg = Twist()
84             msg.linear.x=0.0
85             msg.angular.z=0.0
86             self.vel_pub.publish(msg)
87
88     def target_theta(self, x1, y1, x2, y2):
89         return np.arctan2(y2-y1, x2-x1)
90
91     def normalize_theta(self, theta, goal):
92         if goal[0]==0 and goal[1] == 4:
93             while theta < 0:
94                 theta += 2*np.pi
95             return theta
96         else:
97             return theta
98
99     def odom_callback(self, msg):
100         # get pose = (x, y, theta) from odometry topic
101         quaternion = [msg.pose.pose.orientation.x,msg.pose.pose.orientation.y,\
102             msg.pose.pose.orientation.z, msg.pose.pose.orientation.w]
103         (roll, pitch, yaw) = euler_from_quaternion(quaternion)
104         self.pose.theta = yaw
105         self.pose.x = msg.pose.pose.position.x
106         self.pose.y = msg.pose.pose.position.y

```

```

108     # logging once every 100 times (Gazebo runs at 1000Hz; we save it at 10Hz)
109     self.logging_counter += 1
110     if self.logging_counter == 100:
111         self.logging_counter = 0
112         self.trajectory.append([self.pose.x, self.pose.y]) # save trajectory
113         self.node.get_logger().info("odom: x=" + str(self.pose.x) + \
114                                     "; y=" + str(self.pose.y) + "; theta=" + str(yaw))
115
116 def main(args=None):
117     turtlebot = Turtlebot3()
118
119 if __name__ == '__main__':
120     main()

```

Here are the trajectory plots of both methods.

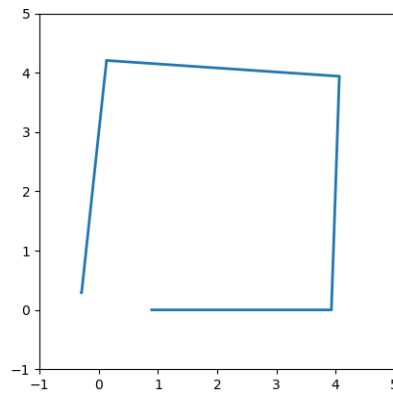


Figure 1: Original

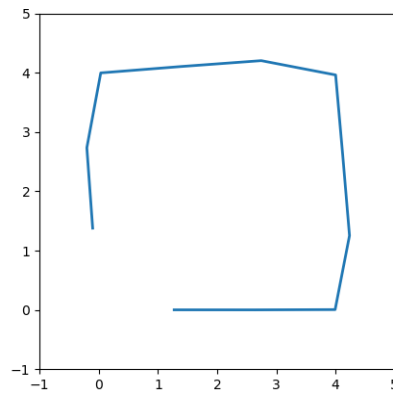


Figure 2: PD

Experiment 1		Experiment 2		Experiment 3	
x	y	x	y	x	y
-0.004501	-0.000886	0.002945	0.000593	0.03445	0.021152
-0.004361	-0.00086	0.002781	0.001219	0.07094	0.035074
-0.003318	-0.001435	0.00525	0.001408	0.043388	0.014404
-0.004816	-0.001344	0.004502	0.000805	0.013611	0.000945
0.006804	-0.002168	0.006219	-0.000725	0.039821	0.001534
-0.003316	-0.001158	0.002494	-0.000789	0.034444	0.041124
-0.005515	-0.000822	-0.003222	0.001415	0.036526	0.019377
-0.003885	-0.000846	0.002978	0.000111	0.010358	0.001728
-0.006976	-0.001976	-0.001769	0.000872	0.024386	-0.00478
0.013538	-0.001094	-0.003788	-0.000206	0.081652	-0.002986

Table 2: Results.

Here are the scatter graphs. The top row are the results from lab 3, and the bottom row are the results from lab 4. When noise is considered, we consider noise as (linear noise, angular noise).

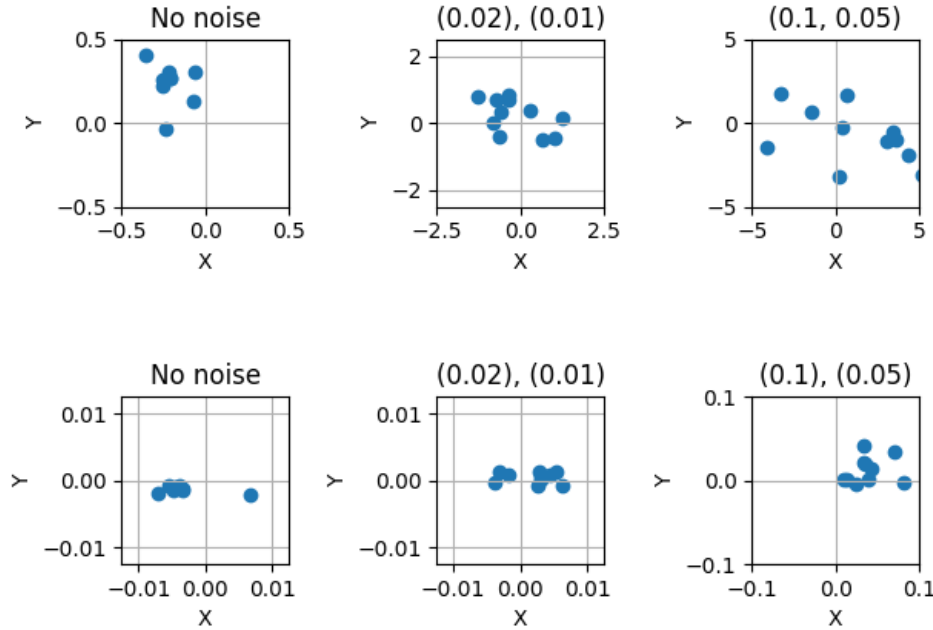


Figure 3: Scatter graphs.

This lab taught me true pain, and that PD is far more successful in keeping a robot less affected by noise. We see that angular speed (rotation angle) is far more significant in affecting my robot, compared to linear speed (uncertainty in drive distance). This is demonstrated by how we use the angle for PD as opposed to linear speed, and it is because one step at an incorrect angle can move the robot away from the target significantly more than if it is approaching the target too fast or slow. Furthermore, error induced by angular speed is exemplified by its much smaller and precise domain of $[-\pi, \pi]$, compared to the real x, y plane affected by linear speed, so it is easier to have more error from a wrong angle than a wrong speed. Under different levels of noise, we can see from the scatter graphs that closed loop control is exceptionally better at shrugging off the noise and getting closer to the target. This is orders of magnitude better as indicated by the scales between noise affecting open loop and closed loop. It is trivial to understand why, as using PD we more regularly update and ensure that our robot is making positive movements toward the target. Therefore open loop control is subjected to the disruption of noise far more. There are the trivial ways that a closed loop control robot may not work, such as the sensors not properly working, therefore we have no access to odometry. Another more interesting way it may fail is if we are working with two closed loop control robots simultaneously, which their respective interactions with environment may cause a loop of how the robot tries to reach their target, constantly going back

and forth. We could develop the PD robot design further to be more precise in several ways. Firstly we could use the original PID equation, using more terms to more accurately account for how the robot corrects itself. Next we could fine-tune through simulation, trial and error, or even machine learning the coefficients for PID, ensuring that we have the most optimal values for a given task. Additionally, we could add realignment algorithms for if the robot gets significantly off track, where we instead aim to return to a known or given path, rather than perhaps futilely trying to reach the next waypoint. Finally we would want to use more advanced sensors which better account for real world granularity, letting us work with greater precision. How we recover from motion drift is seen with PD, proportionally moving the robot back to the original path, but we can improve it with a method of calibration. Depending on our sensors, defining constants while the robot moves that can be used as references to what is correct and what motion should entail, could let us then choose better motion options. We may also use a Kalman filter to reduce uncertainties and move such that noise has a minimised impact on our robot.

5 Task 5

5.1 Task 5-1

Code to extract laser data:

```

1 import rclpy
2 from rclpy.node import Node
3 from sensor_msgs.msg import LaserScan
4
5 class Laser(Node):
6     def __init__(self):
7         super().__init__('laser')
8         self.subscription_ = self.create_subscription(LaserScan, 'scan', self.scan_callback, 10)
9
10        self.subscription_ # prevent unused variable warning
11        self.get_logger().info('Laser has been initialized.')
12
13        def scan_callback(self, msg: LaserScan):
14            string = str(msg.ranges[0]) + ", " + str(msg.ranges[89]) + ", " + str(msg.ranges[179]) + ", " + str(
15                msg.ranges[269])
16            self.get_logger().info(string)
17
18 def main(args=None):
19     rclpy.init(args=args)
20     laser = Laser()
21     rclpy.spin(laser)
22     laser.destroy_node()
23     rclpy.shutdown()
24
25 if __name__ == '__main__':
26     main()

```

5.2 Task 5-2

Code for open loop object avoidance:

```

1 import rclpy
2 from rclpy.node import Node
3 from sensor_msgs.msg import LaserScan
4 from geometry_msgs.msg import Twist
5
6 class Avoid(Node):
7     def __init__(self):
8         super().__init__('laser')
9         self.subscription_ = self.create_subscription(LaserScan, 'scan', self.scan_callback, 10)
10        self.publisher_ = self.create_publisher(Twist, 'cmd_vel', 10)
11        self.subscription_ # prevent unused variable warning
12        self.get_logger().info('Avoid has been initialized.')
13
14        def scan_callback(self, msg: LaserScan):
15            new_msg = Twist()
16            if msg.ranges[0] < 0.3:
17                new_msg.linear.x = 0.0
18                self.publisher_.publish(new_msg)

```

```

19         else:
20             new_msg.linear.x = 0.4
21             self.publisher_.publish(new_msg)
22
23 def main(args=None):
24     rclpy.init(args=args)
25     avoid = Avoid()
26     rclpy.spin(avoid)
27     avoid.destroy_node()
28     rclpy.shutdown()
29
30 if __name__ == '__main__':
31     main()

```

This code takes a very naive approach to getting the robot to stop, where it constantly takes in the laser's scan message, and if the distance from the front laser, indicated by `msg.ranges[0]`, is less than our threshold 0.3, then we immediately stop and kill all linear velocity. Otherwise, we move forward as intended, and move at 0.4 speed.

Code for closed loop object avoidance:

```

1 import rclpy
2 from rclpy.node import Node
3 from sensor_msgs.msg import LaserScan
4 from geometry_msgs.msg import Twist
5
6 class Controller:
7     def __init__(self, P=0.0, D=0.0, set_point=0):
8         self.Kp = P
9         self.Kd = D
10        self.set_point = set_point # reference (desired value)
11        self.previous_error = 0
12
13    def update(self, current_value):
14        # calculate P_term and D_term
15        error = self.set_point - current_value
16        P_term = self.Kp * error
17        D_term = self.Kd * (error - self.previous_error)
18        self.previous_error = error
19        return P_term + D_term
20
21    def setPoint(self, set_point):
22        self.set_point = set_point
23        self.previous_error = 0
24
25    def setPD(self, P=0.0, D=0.0):
26        self.Kp = P
27        self.Kd = D
28
29 class Avoid(Node):
30     def __init__(self):
31         super().__init__('laser')
32         self.subscription_ = self.create_subscription(LaserScan, 'scan', self.scan_callback, 10)
33         self.publisher_ = self.create_publisher(Twist, 'cmd_vel', 10)
34         self.subscription_ # prevent unused variable warning
35         self.get_logger().info('Avoid has been initialized.')
36
37         self.controller = Controller(0.8, 0.6, 0.3)
38
39     def scan_callback(self, msg: LaserScan):
40
41         new_msg = Twist()
42         speed = min(self.controller.update(msg.ranges[0]), 0.4)
43         if speed > 0.0:
44             new_msg.linear.x = speed
45         else:
46             new_msg.linear.x = 0.0
47
48         self.publisher_.publish(new_msg)
49
50 def main(args=None):
51     rclpy.init(args=args)
52     avoid = Avoid()

```

```

53     rclpy.spin(avoid)
54     avoid.destroy_node()
55     rclpy.shutdown()
56
57 if __name__ == '__main__':
58     main()

```

This code instead now will gradually reach the stopping threshold of 0.3 using the Controller class previously implemented, where we use PD to slow our approach toward a detected wall. We go either the slowing down speed, or the speed of 0.4. If we have stopped, and therefore our calculated PD is below 0, we simply kill linear velocity and set that to 0. Our approach to determine the coefficients was found through trial and error, finding the values that slowed the robot at an appropriate rate. We just tested in the standard range of 0.1 and 0.9 for both k_d and k_p .

If the objects that we are trying avoid colliding with are moving, as in they have non zero velocity, then our current methods will not be appropriate as we may slow down too fast or too slow to appropriately react to the object. We can have a variable threshold distance to account for this, where fast moving objects must result in a greater threshold distance, as then our PD will more rapidly slow us down. We would want to make threshold distance dependent on both the velocity of the robot and the object to the best of the sensing ability. If we were to just use lasers, we may be limited to use the ranges to approximate how fast an object may or may not be moving toward us. If we have more sensors, we may have a better idea or even a mapping of objects in the area, which subsequently would let us predict where they may be in x number of time steps, which could influence our stopping distance.